

Hello, Android: Deep Dive

Introduction to Android development with Xamarin

Hello, Android Deep Dive

In the [Hello, Android Quickstart](#), you built and ran your first Xamarin.Android application. Now it's time to develop a deeper understanding of how Android applications work so that you can build more sophisticated programs. This guide reviews the steps that you took in the Hello, Android walkthrough so that you can understand what you did and begin to develop a fundamental understanding of Android application development.

This guide will touch upon the following topics:

[[ide name="xs"]]

- **Introduction to Visual Studio for Mac** – Introduction to Xamarin Studio and creating a new Xamarin.Android application.
- **Anatomy of a Xamarin.Android Application** – Tour of the essential parts of a Xamarin.Android application.
- **App Fundamentals and Architecture Basics** – Introduction to Activities, the Android Manifest, and the general flavor of Android development.
- **User Interface (UI)** – Creating user interfaces with the Android Designer.
- **Activities and the Activity Lifecycle** – An introduction to the Activity Lifecycle and wiring up the user interface in code.
- **Testing, Deployment, and Finishing Touches** – Complete your application with advice on testing, deployment, generating artwork, and more.

[/ide]

[[ide name="vs"]]

- **Introduction to Visual Studio** – Introduction to Visual Studio and creating a new Xamarin.Android application.

- **Anatomy of a Xamarin.Android Application** - Tour of the essential parts of a Xamarin.Android application.
- **App Fundamentals and Architecture Basics** – Introduction to Activities, the Android Manifest, and the general flavor of Android development.
- **User Interface (UI)** – Creating user interfaces with the Android Designer.
- **Activities and the Activity Lifecycle** – An introduction to the Activity Lifecycle and wiring up the user interface in code.
- **Testing, Deployment, and Finishing Touches** – Complete your application with advice on testing, deployment, generating artwork, and more.

[[/ide]]

This guide helps you develop the skills and knowledge required to build a single-screen Android application. After you work through it, you should understand the different parts of a Xamarin.Android application and how they fit together.

[[ide name="xs"]]

Introduction to Visual Studio for Mac

Visual Studio for Mac is a free, open-source IDE similar to Visual Studio. It features a fully integrated visual designer, a text editor complete with refactoring tools, an assembly browser, source code integration, and more. In this guide, you'll learn to use some basic Visual Studio for Mac features. If you're new to Visual Studio for Mac, you may want to check out the more in-depth [Introduction to Visual Studio for Mac](#).

Visual Studio for Mac follows the Visual Studio practice of organizing code into *Solutions* and *Projects*. A Solution is a container that can hold one or more Projects. A Project can be an application (such as for iOS or Android), a supporting library, a test application, and more. In the **Phoneword** app, you added a new Android Project using the **Android Application** template to the **Phoneword** Solution created in the [Hello, Android](#) guide.

[[/ide]]

[[ide name="vs"]]

Introduction to Visual Studio

Visual Studio is a powerful IDE from Microsoft. It features a fully integrated visual designer, a text editor that includes refactoring tools, an assembly browser, source code integration, and more. In this guide you'll learn to use some basic Visual Studio features with the Xamarin plug-in.

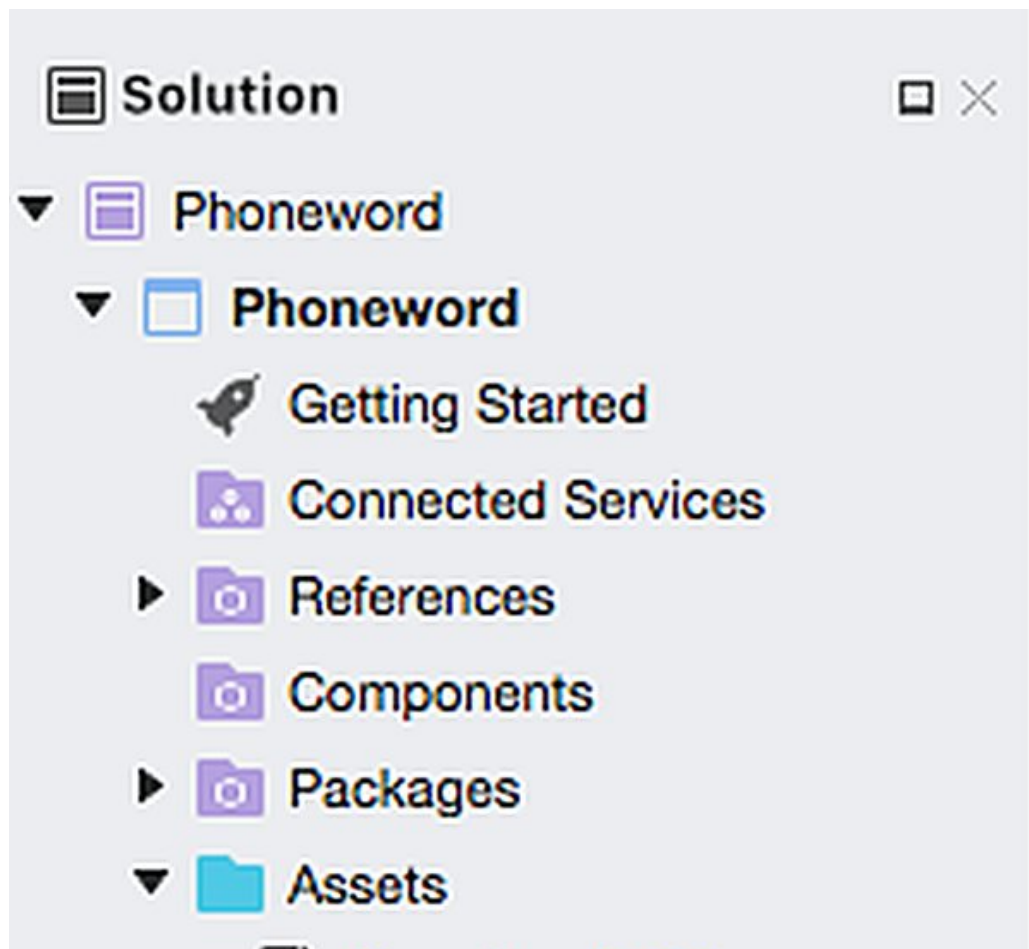
Visual Studio organizes code into *Solutions* and *Projects*. A Solution is a container that can hold one or more Projects. A Project can be an application (such as for iOS or Android), a supporting library, a test application, and more. In the **Phoneword** app, you added a new Android Project using the **Android Application** template to the **Phoneword** Solution created in the [Hello, Android](#) guide.

[[/ide]]

Anatomy of a Xamarin.Android Application

[[ide name="xs"]]

The following screenshot lists the Solution's contents. This is the Solution Pad, which contains the directory structure and all of the files associated with the Solution:



 AboutAssets.txt

▼ Properties

 AndroidManifest.xml

 AssemblyInfo.cs

▼ Resources

 drawable

▼ layout

 Main.axml

▼ mipmap-hdpi

 Icon.png

▼ mipmap-mdpi

 Icon.png

▼ mipmap-xhdpi

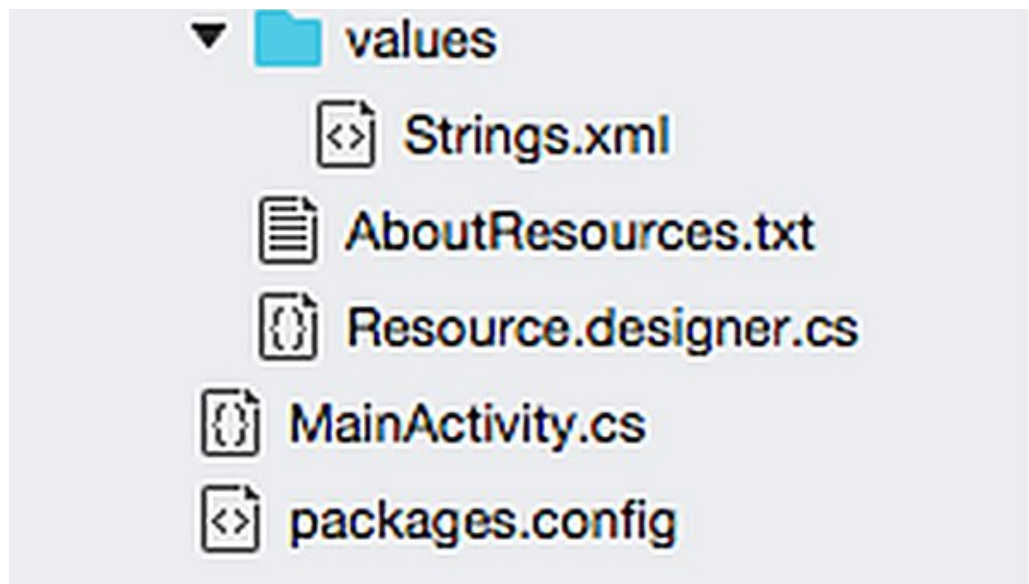
 Icon.png

▼ mipmap-xxhdpi

 Icon.png

▼ **mipmap-xxxhdpi**

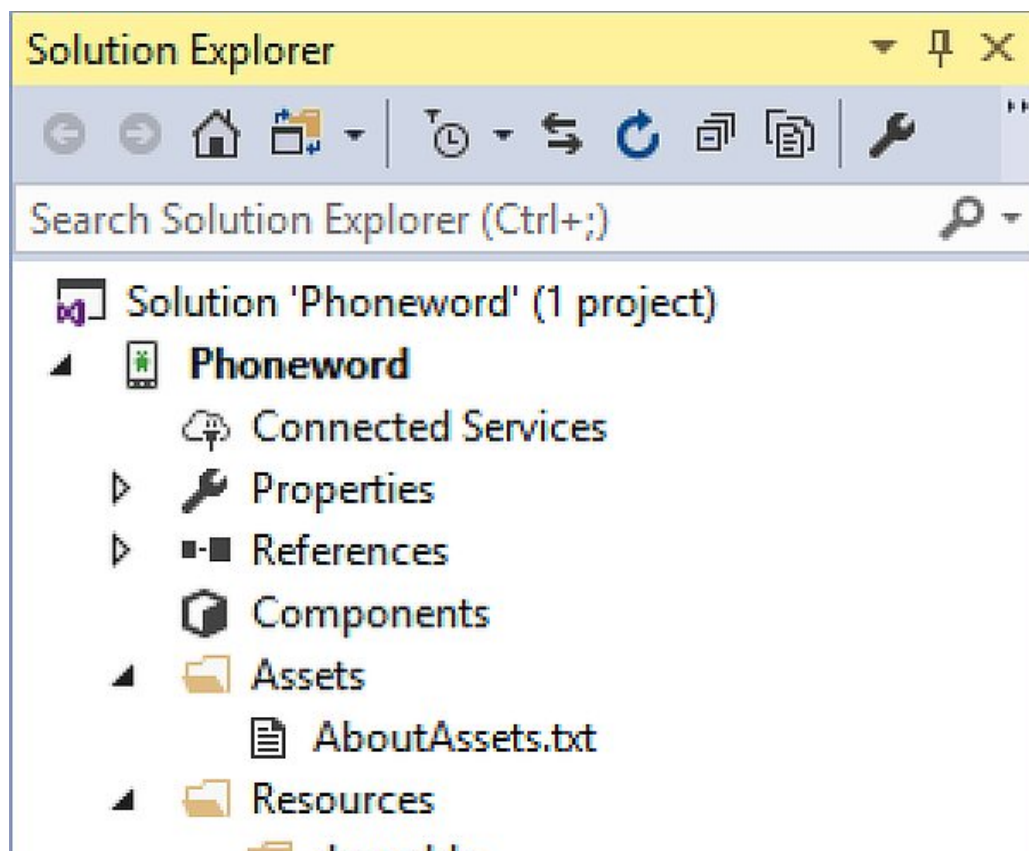
 Icon.png

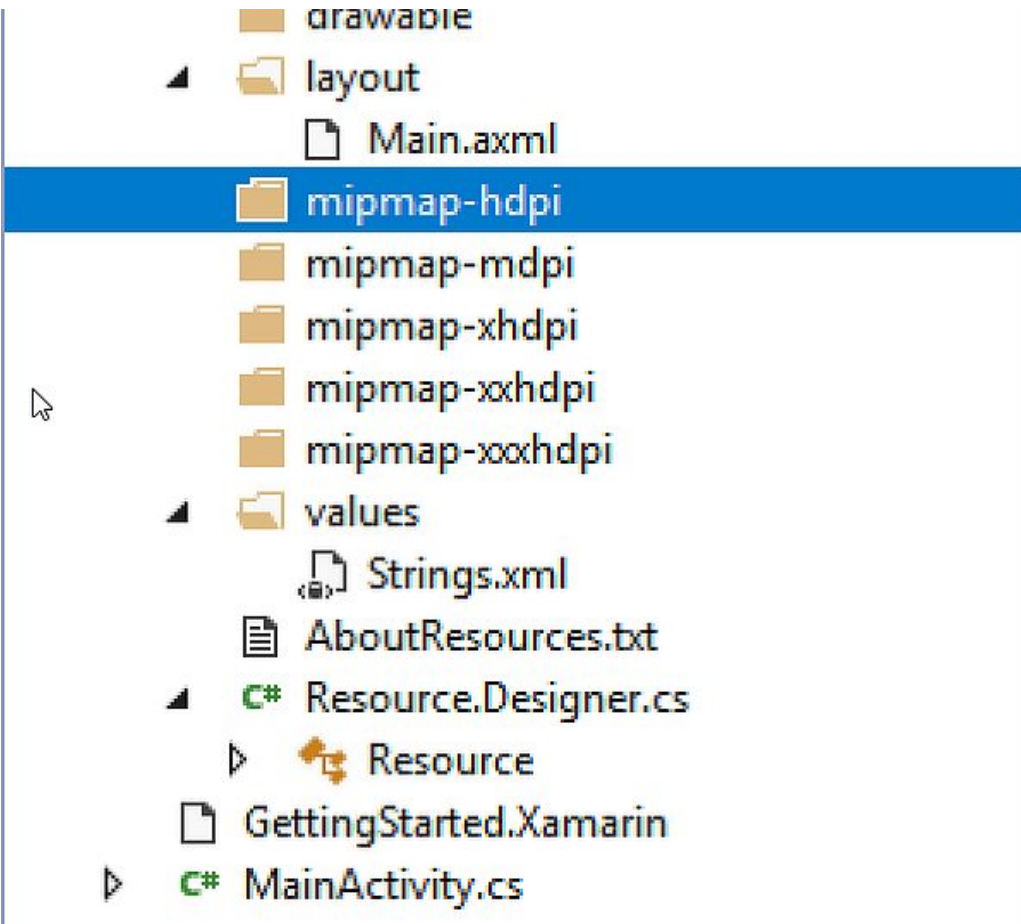


[[/ide]]

[[ide name="vs"]]

The following screenshot lists the Solution's contents. This is the Solution Explorer, which contains the directory structure and all of the files associated with the Solution:





[[/ide]]

A Solution called **Phoneword** was created and the Android project **Phoneword** was placed inside of it.

Look at the items inside the Project to see each folder and its purpose:

- **Properties** – Contains the [AndroidManifest.xml](#) file that describes all of the requirements for the Xamarin.Android application, including name, version number, and permissions. The **Properties** folder also houses [AssemblyInfo.cs](#), a .NET assembly metadata file. It is a good practice to fill this file with some basic information about your application.
- **References** – Contains the assemblies required to build and run the application. If you expand the References directory, you'll see references to .NET assemblies such as [System](#), System.Core, and [System.Xml](#), as well as a reference to Xamarin's Mono.Android assembly.
- **Assets** – Contains the files the application needs to run including fonts, local data files, and text files. Files included here are accessible through the generated `Assets` class. For more information on Android Assets, see the Xamarin [Using Android Assets](#) guide.

- **Resources** – Contains application resources such as strings, images, and layouts. You can access these resources in code through the generated `Resource` class. The [Android Resources](#) guide provides more details about the **Resources** directory. The application template also includes a concise guide to Resources in the **AboutResources.txt** file.

Resources

The **Resources** directory contains four folders named **drawable**, **layout**, **mipmap** and **values**, as well as a file named **Resource.designer.cs**.

The items are summarized in the table below:

- **drawable** – The drawable directories house [drawable resources](#) such as images and bitmaps.
- **mipmap** – The mipmap directory holds drawable files for different launcher icon densities. In the default template, the drawable directory houses the application icon file, **Icon.png**.
- **layout** – The layout directory contains *Android designer files* (.xml) that define the user interface for each screen or Activity. The template creates a default layout called **Main.xml**.
- **values** – This directory houses XML files that store simple values such as strings, integers, and colors. The template creates a file to store string values called **Strings.xml**.
- **Resource.designer.cs** – Also known as the `Resource` class, this file is a partial class that holds the unique IDs assigned to each resource. It is automatically created by the Xamarin.Android tools and is regenerated as necessary. This file should not be manually edited, as Xamarin.Android will overwrite any manual changes made to it.

App Fundamentals and Architecture Basics

Android applications do not have a single entry point; that is, there is no single line of code in the application that the operating system calls to start the application. Instead, an application starts when Android instantiates one of its classes, during which time Android loads the entire application's process into memory.

This unique feature of Android can be extremely useful when designing complicated applications or interacting with the Android operating system. However, these options also make Android complex when dealing with a basic scenario like the **Phoneword** application. For this reason, exploration of Android architecture is split in two. This guide dissects an application that uses the most common entry point for an Android app: the first screen. In [Hello, Android Multiscreen](#), the full complexities of Android architecture are

explored as different ways to launch an application are discussed.

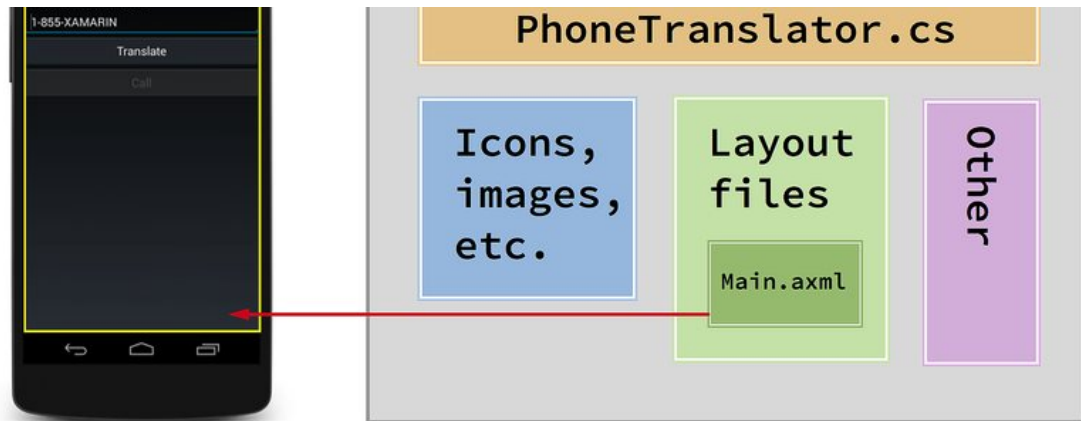
Phoneword Scenario - Starting with an Activity

When you open the **Phoneword** application for the first time in an emulator or device, the operating system creates the first *Activity*. An Activity is a special Android class that corresponds to a single application screen, and it is responsible for drawing and powering the user interface. When Android creates an application's first Activity, it loads the entire application:



Since there is no linear progression through an Android application (you can launch the application from several points), Android has a unique way of keeping track of what classes and files make up an application. In the **Phoneword** example, all the parts that make up the application are registered with a special XML file called the **Android Manifest**. The role of the **Android Manifest** is to keep track of an application's contents, properties, and permissions and to disclose them to the Android operating system. You can think of the **Phoneword** application as a single Activity (screen) and a collection of resource and helper files tied together by the Android Manifest file, as illustrated by the diagram below:





Activity (Screen)



Android Manifest

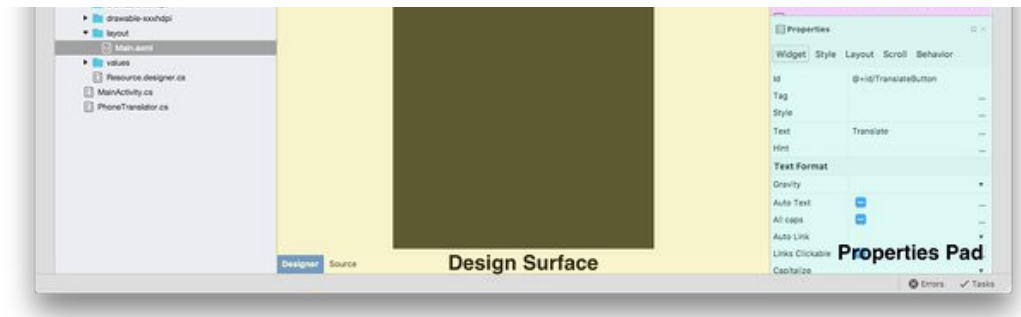
The next few sections explore the relationships between the various parts of the **Phoneword** application; this should provide you with a better understanding of the diagram above. This exploration begins with the user interface as it discusses the Android designer and layout files.

User Interface

`Main.xml` is the user interface layout file for the first screen in the application. The `.xml` indicates that this is an Android designer file (AXML stands for *Android XML*). The name *Main* is arbitrary from Android's point of view – the layout file could have been named something else. When you open **Main.xml** in the IDE, it brings up the visual editor for Android layout files called the *Android Designer*:

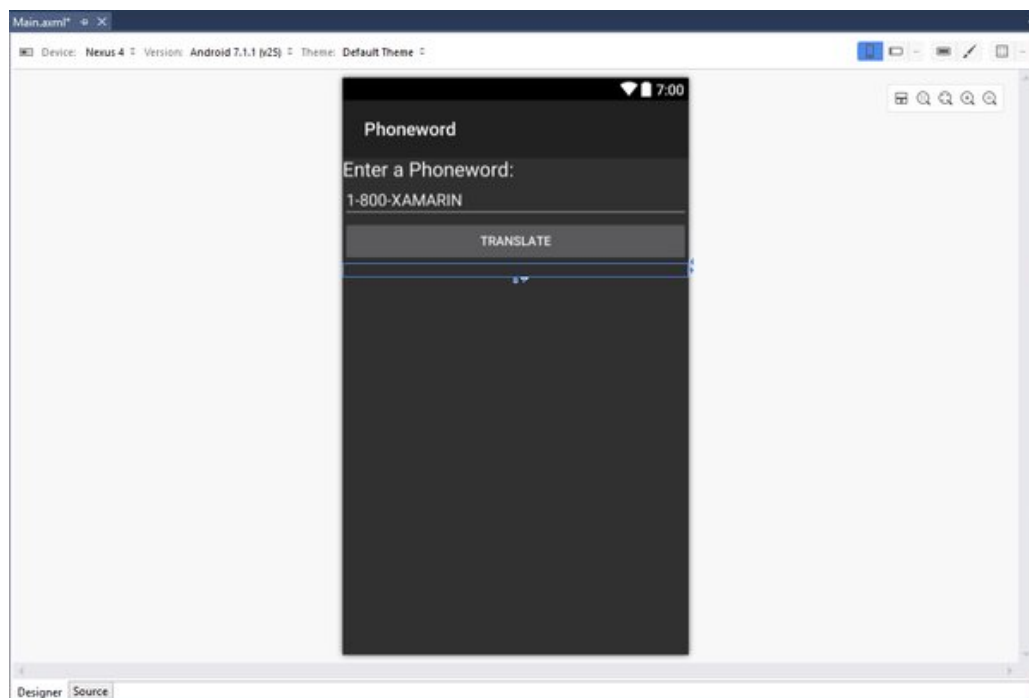
[[ide name="xs"]]





[[/ide]]

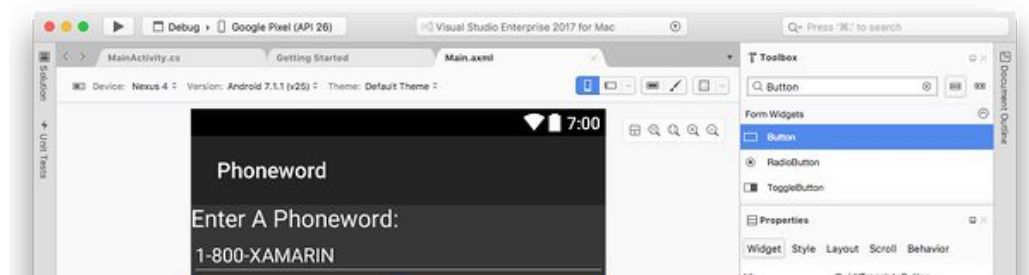
[[ide name="vs"]]

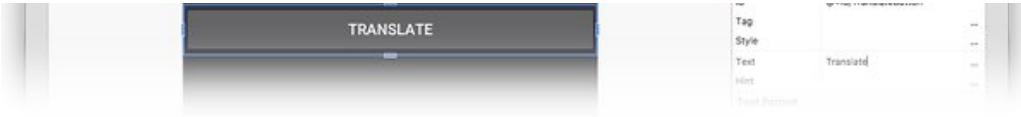


[[/ide]]

In the **Phoneword** app, the **TranslateButton**'s ID is set to @+id/TranslateButton:

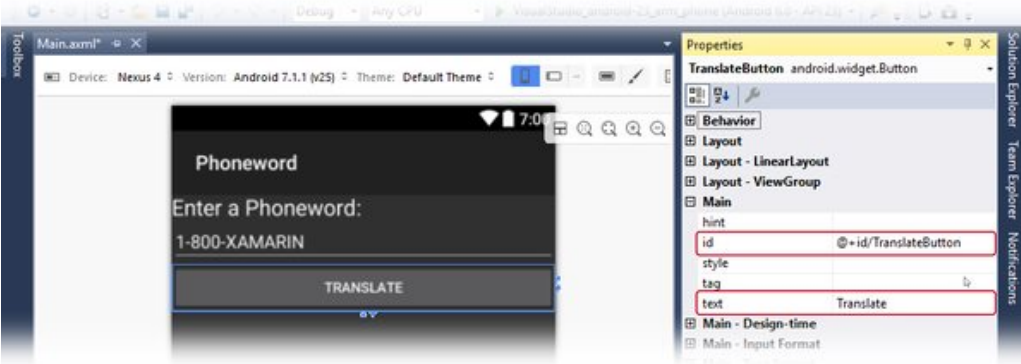
[[ide name="xs"]]





[[/ide]]

[[ide name="vs"]]



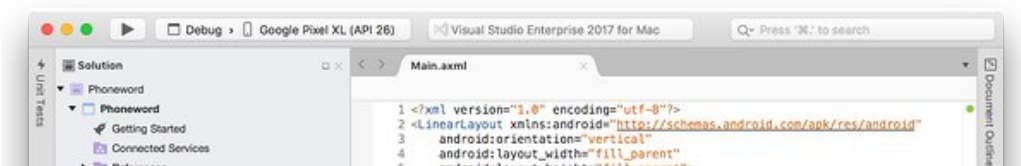
[[/ide]]

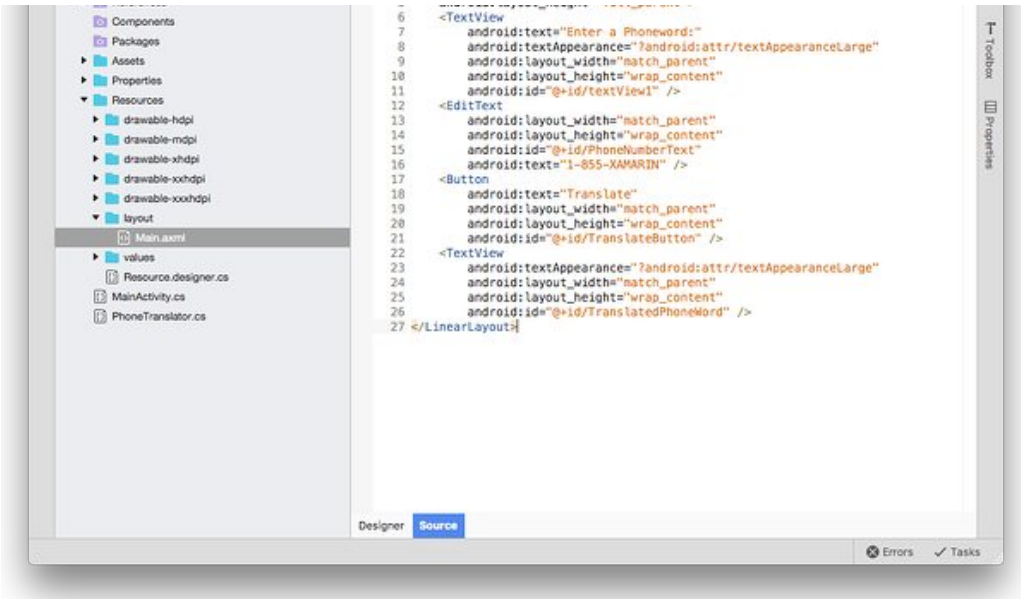
When you set the `id` property of the **TranslateButton**, the Android Designer maps the **TranslateButton** control to the `Resource` class and assigns it a *resource ID* of `TranslateButton`. This mapping of visual control to class makes it possible to locate and use the **TranslateButton** and other controls in app code. This will be covered in more detail when you break apart the code that powers the controls. All you need to know for now is that the code representation of a control is linked to the visual representation of the control in the designer via the `id` property.

Source View

Everything defined on the design surface is translated into XML for Xamarin.Android to use. The Android Designer provides a source view that contains the XML that was generated from the visual designer. You can view this XML by switching to the **Source** panel in the lower left of the designer view, as illustrated by the screenshot below:

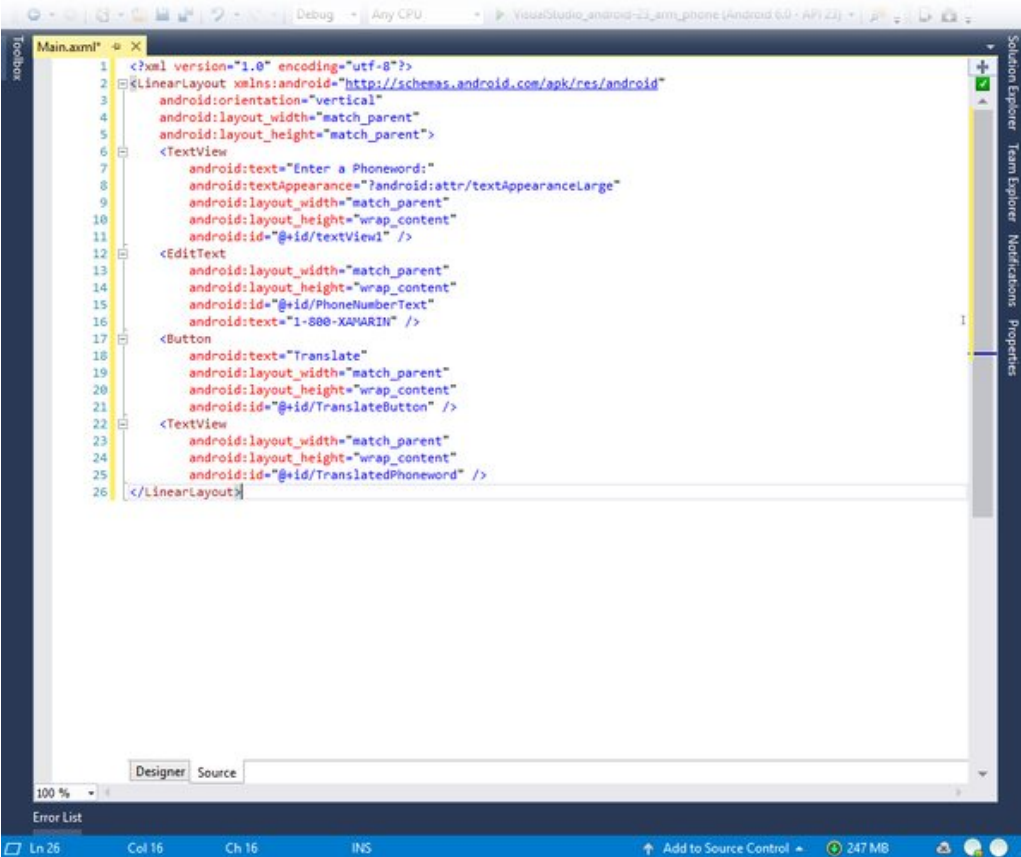
[[ide name="xs"]]





[[/ide]]

[[ide name="vs"]]



[[/ide]]

This XML source code should contain the **Text (Large)**, **Plain Text**, and the two **Button** elements. For a more in-depth tour of the Android Designer, refer to the Xamarin Android [Designer Overview](#) guide.

The tools and concepts behind the visual part of the user interface have now been covered. Next, it's time to jump into the code that powers the user interface as Activities and the Activity Lifecycle are explored.

Activities and the Activity Lifecycle

The `Activity` class contains the code that powers the user interface. The Activity is responsible for responding to user interaction and creating a dynamic user experience. This section introduces the `Activity` class, discusses the Activity Lifecycle, and dissects the code that powers the user interface in the **Phoneword** application.

Activity Class

The **Phoneword** application has only one screen (Activity). The class that powers the screen is called `MainActivity` and lives in the **MainActivity.cs** file. The name `MainActivity` has no special significance in Android – although the convention is to name the first Activity in an application `MainActivity`, Android does not care if it is named something else.

When you open **MainActivity.cs**, you can see that the `MainActivity` class is a *subclass* of the `Activity` class, and that the Activity is adorned with the [Activity](#) attribute:

```
[Activity (Label = "Phone Word", MainLauncher = true)]
public class MainActivity : Activity
{
    ...
}
```

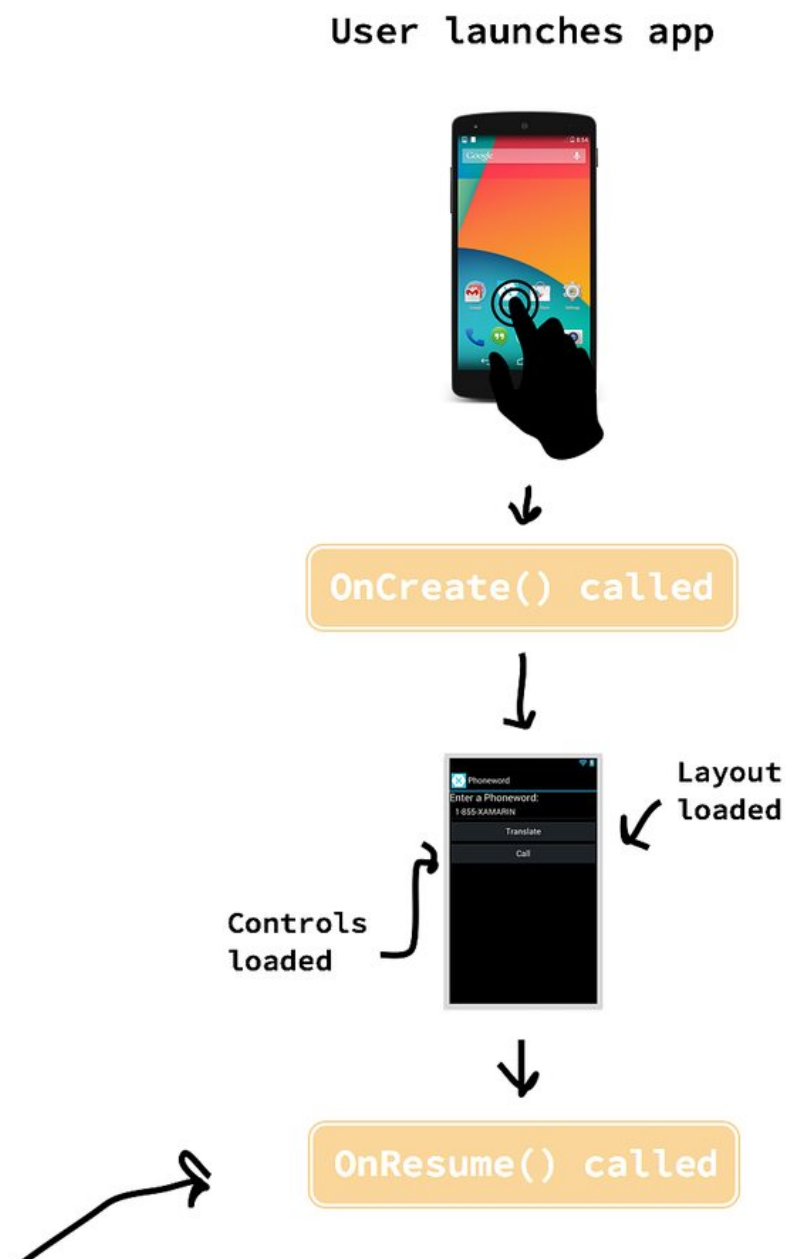
The `Activity` Attribute registers the Activity with the Android Manifest; this lets Android know that this class is part of the **Phoneword** application managed by this manifest. The `Label` property sets the text that will be displayed at the top of the screen.

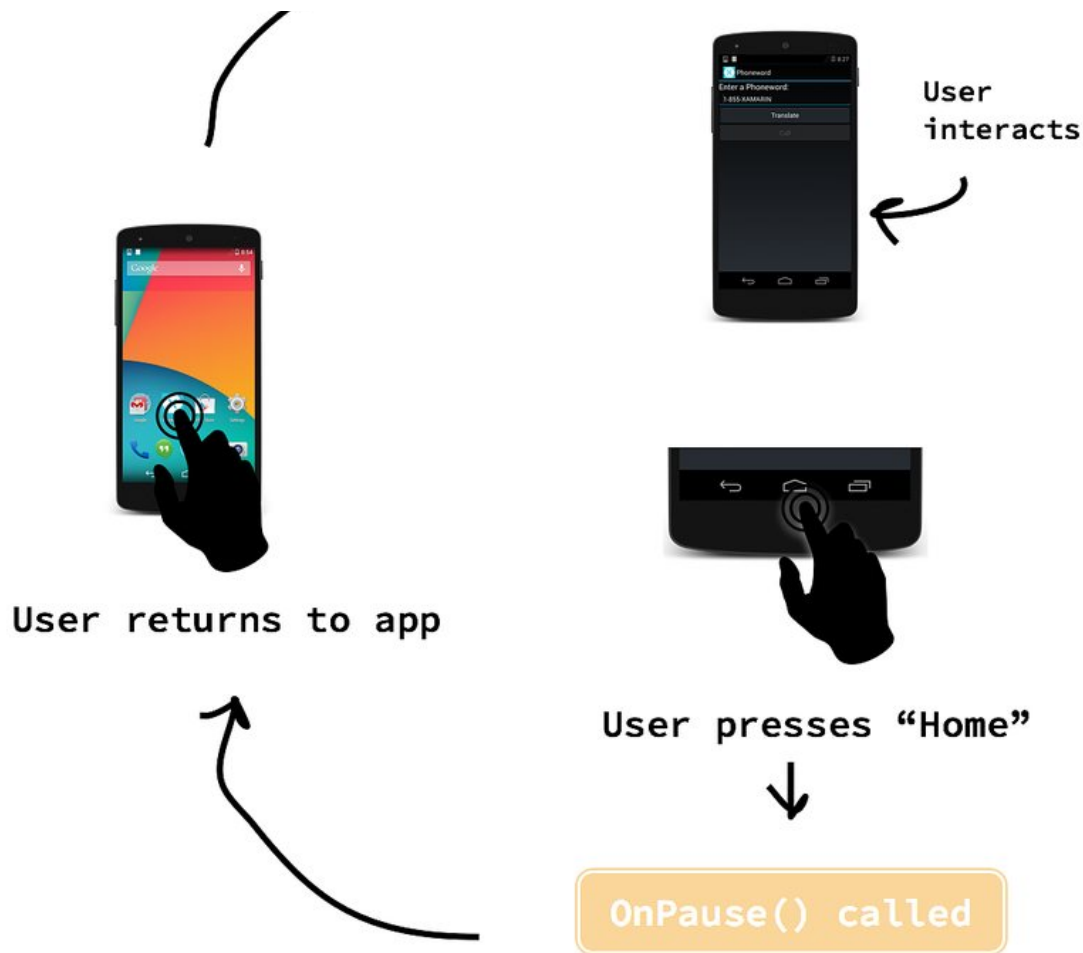
The `MainLauncher` property tells Android to display this Activity when the application starts up. This property becomes important as you add more Activities (screens) to the application as explained in the [Hello, Android Multiscreen](#) guide.

Now that the basics of `MainActivity` have been covered, it's time to dive deeper into the Activity code by introducing the *Activity Lifecycle*.

Activity Lifecycle

In Android, Activities go through different stages of a lifecycle depending on their interactions with the user. Activities can be created, started and paused, resumed and destroyed, and so on. The `Activity` class contains methods that the system calls at certain points in the screen's lifecycle. The following diagram illustrates a typical life of an Activity as well as some of the corresponding lifecycle methods:





By overriding `Activity` lifecycle methods, you can control how the Activity loads, how it reacts to the user, and even what happens after it disappears from the device screen. For example, you can override the lifecycle methods in the diagram above to perform some important tasks:

- **OnCreate** – Creates views, initializes variables, and performs other prep work that must be done before the user sees the Activity. This method is called only once when the Activity is loaded into memory.
- **OnResume** – Performs any tasks that must happen every time the Activity returns to the device screen.
- **OnPause** – Performs any tasks that must happen every time the Activity leaves the device screen.

When you add custom code to a lifecycle method in the `Activity`, you *override* that lifecycle method's *base implementation*. You tap into the existing lifecycle method (which has some code already attached to it), and you extend that method with your own code. You call the base implementation from inside your

method to ensure that the original code runs before your new code. An example of this is illustrated in the next section.

The Activity Lifecycle is an important and complex part of Android. If you'd like to learn more about Activities after you finish the *Getting Started* series, read the [Activity Lifecycle](#) guide. In this guide, the next focus is the first stage of the Activity Lifecycle, `onCreate`.

onCreate

Android calls the Activity's `onCreate` method when it creates the Activity (before the screen is presented to the user). You can override the `onCreate` lifecycle method to create views and prepare your Activity to meet the user:

```
protected override void onCreate (Bundle bundle)
{
    base.onCreate (bundle);

    // Set our view from the "main" layout resource
    setContentView (Resource.Layout.Main);
    // Additional setup code will go here
}
```

In the **Phoneword** app, the first thing to do in `onCreate` is load the user interface created in the Android Designer. To load the UI, call `setContentView` and pass it the *resource layout name* for the layout file: `Main.xml`. The layout is located at `Resource.Layout.Main`:

```
setContentView (Resource.Layout.Main);
```

When `MainActivity` starts up, it creates a view that is based on the contents of the **Main.xml** file. Note that the layout file name is matched to the Activity name – *Main.xml* is the layout for *MainActivity*. This isn't required from Android's point of view, but as you begin to add more screens to the application, you'll find that this naming convention makes it easier to match the code file to the layout file.

After the layout file is prepared, you can start looking up controls. To look up a control, call `findViewById` and pass in the resource ID of the control:

```
EditText phoneNumberText = findViewById<EditText>(Resource.Id.PhoneNumberText);
Button translateButton = findViewById<Button>(Resource.Id.TranslateButton);
TextView translatedPhoneWord = findViewById<TextView>
(Resource.Id.TranslatedPhoneWord);
```

Now that you have references to the controls in the layout file, you can start programming them to respond to user interaction.

Responding to User Interaction

In Android, the `Click` event listens for the user's touch. In this app, the `Click` event is handled with a lambda, but a delegate or a named event handler could be used instead. The final **TranslateButton** code resembled the following:

```
translateButton.Click += (sender, e) =>
{
    // Translate user's alphanumeric phone number to numeric
    translatedNumber = PhonewordTranslator.ToNumber(phoneNumberText.Text);
    if (string.IsNullOrEmpty(translatedNumber))
    {
        translatedPhoneWord.Text = string.Empty;
    }
    else
    {
        translatedPhoneWord.Text = translatedNumber;
    }
};
```

Testing, Deployment, and Finishing Touches

Both Visual Studio for Mac and Visual Studio provide many options for testing and deploying an application. This section covers debugging options, demonstrates testing applications on a device, and introduces tools for creating custom app icons for different screen densities.

Debugging Tools

Issues in application code can be difficult to diagnose. To help diagnose complex code issues, you can [Set a Breakpoint](#), [Step Through Code](#), or [Output Information to the Log Window](#).

Deploy to a Device

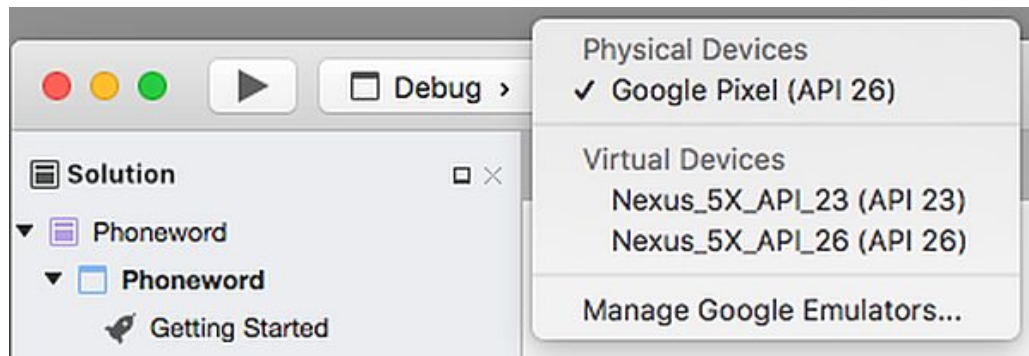
The emulator is a good start for deploying and testing an application, but users will not consume the final

app in an emulator. It's a good practice to test applications on a real device early and often.

Before an Android device can be used for testing applications, it needs to be configured for development. The [Set Up Device for Development](#) guide provides thorough instructions on getting a device ready for development.

[[ide name="xs"]]

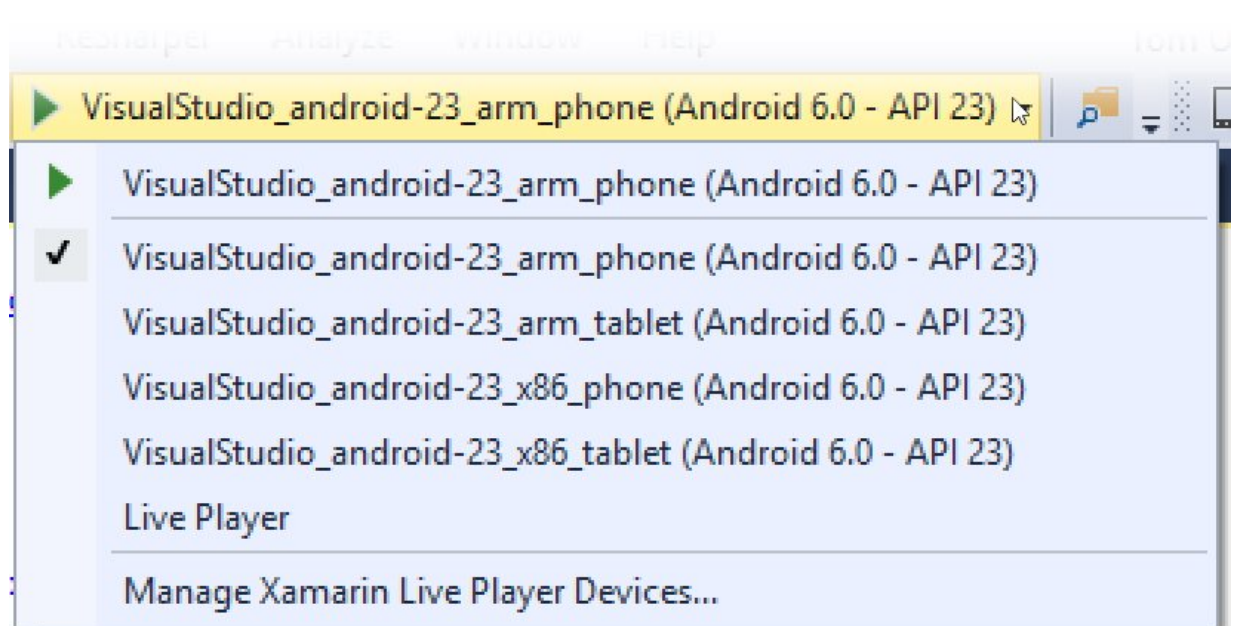
After the device is configured, you can deploy to it by plugging it in, pressing **Start (Play)**, selecting it from the **Select Device** dialog, and pressing **OK**:



[/ide]

[[ide name="vs"]]

After the device is configured, you can deploy to it by plugging it in, selecting it from the **Select Device** dialog, and starting the application:



[[/ide]]

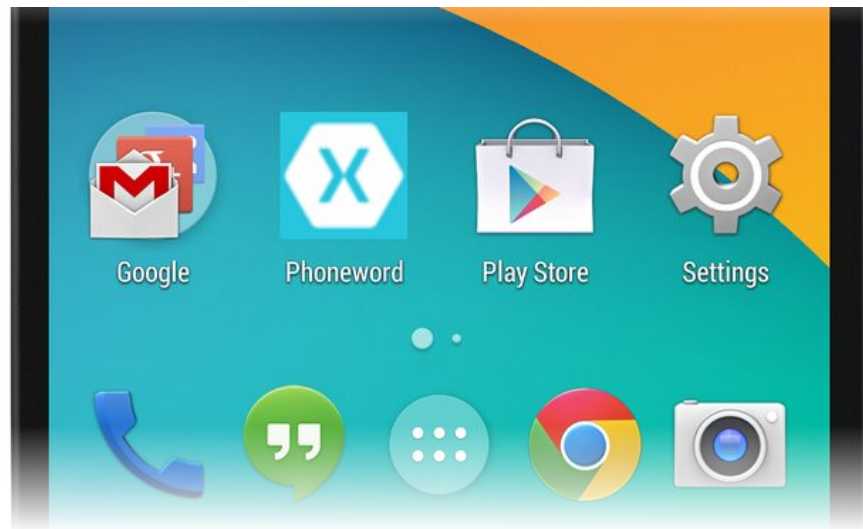
This launches the application on the device:



Set Icons for Different Screen Densities

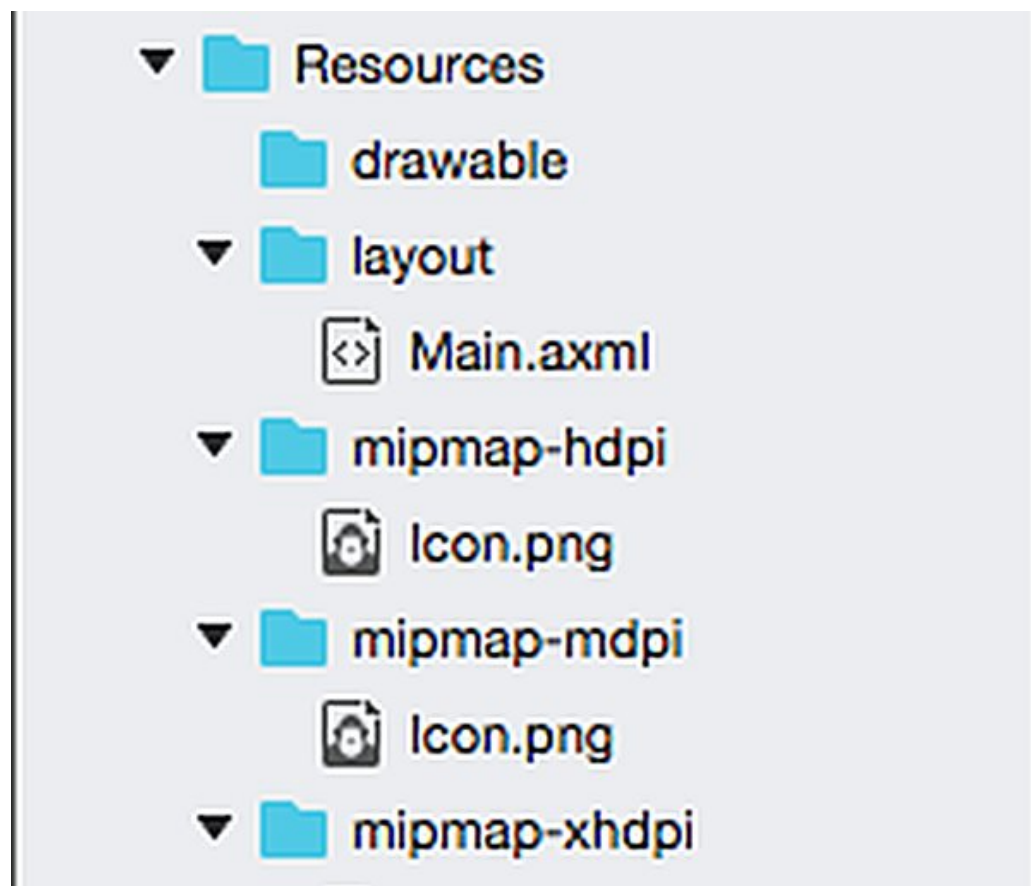
Android devices come in different screen sizes and resolutions, and not all images look good on all screens. For example, here is a screenshot of a low-density icon on a high-density Nexus 5. Notice how blurry it is compared to the surrounding icons:

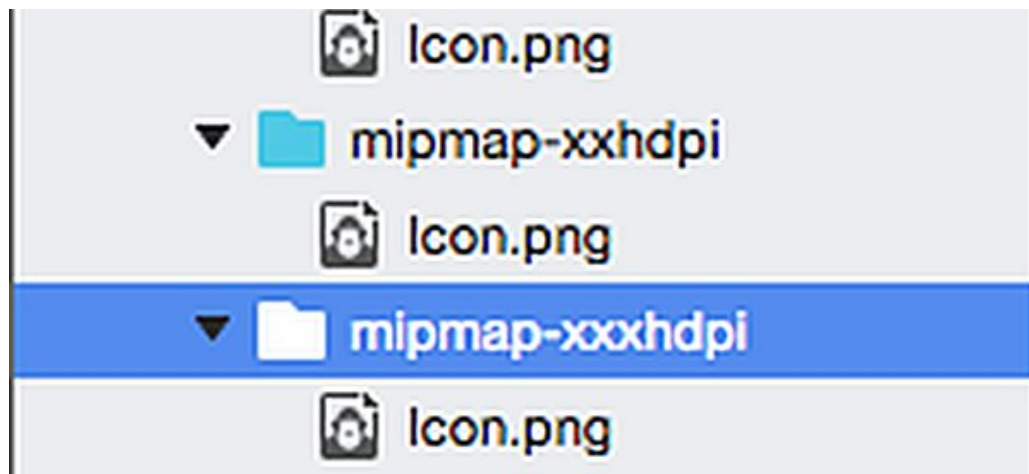




To account for this, it is good practice to add icons of different resolutions to the **Resources** folder. Android provides different versions of the **mipmap** folder to handle launcher icons of different densities, *mdpi* for medium, *hdpi* for high, and *xhdpi*, *xxhdpi*, *xxxhdpi* for very high density screens. Icons of varying sizes are stored in the appropriate **mipmap**- folders:

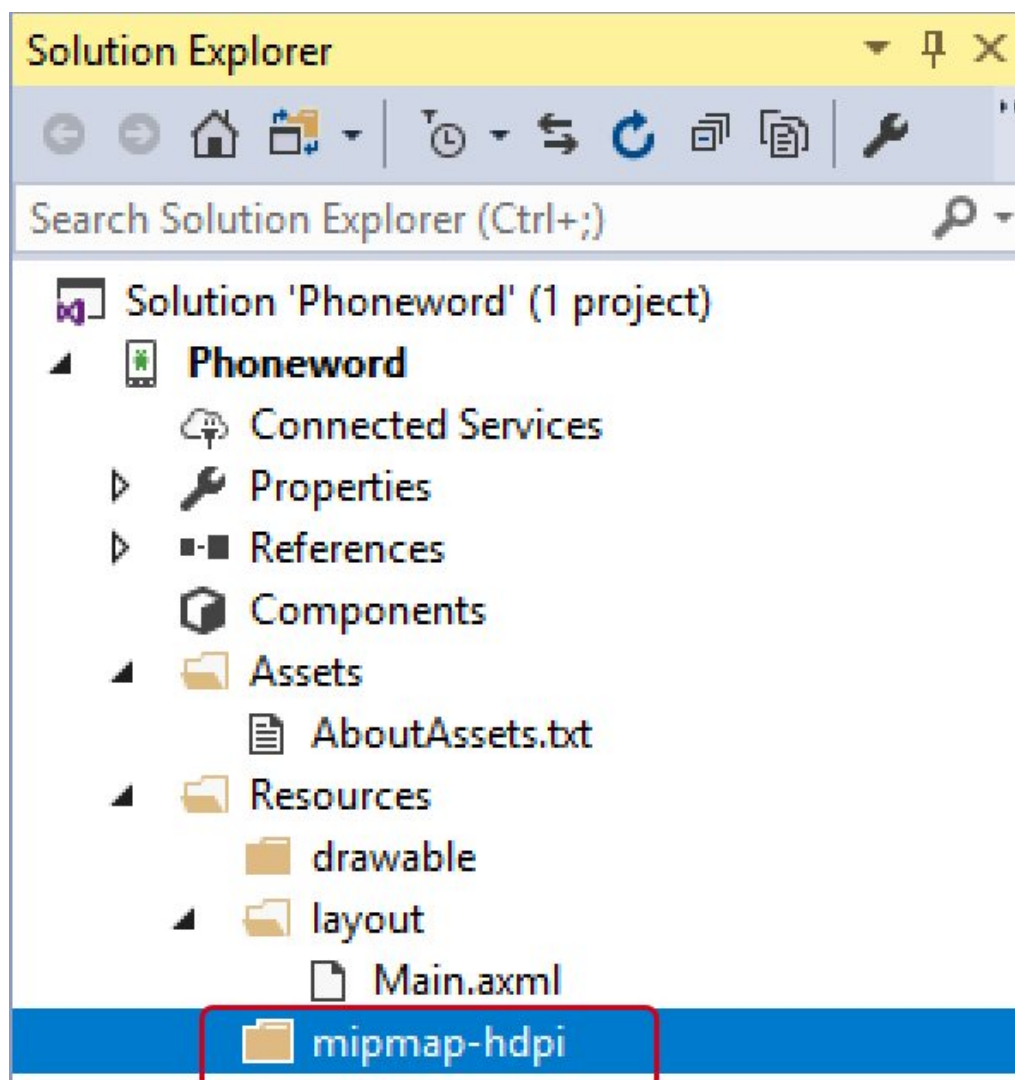
[[ide name="xs"]]

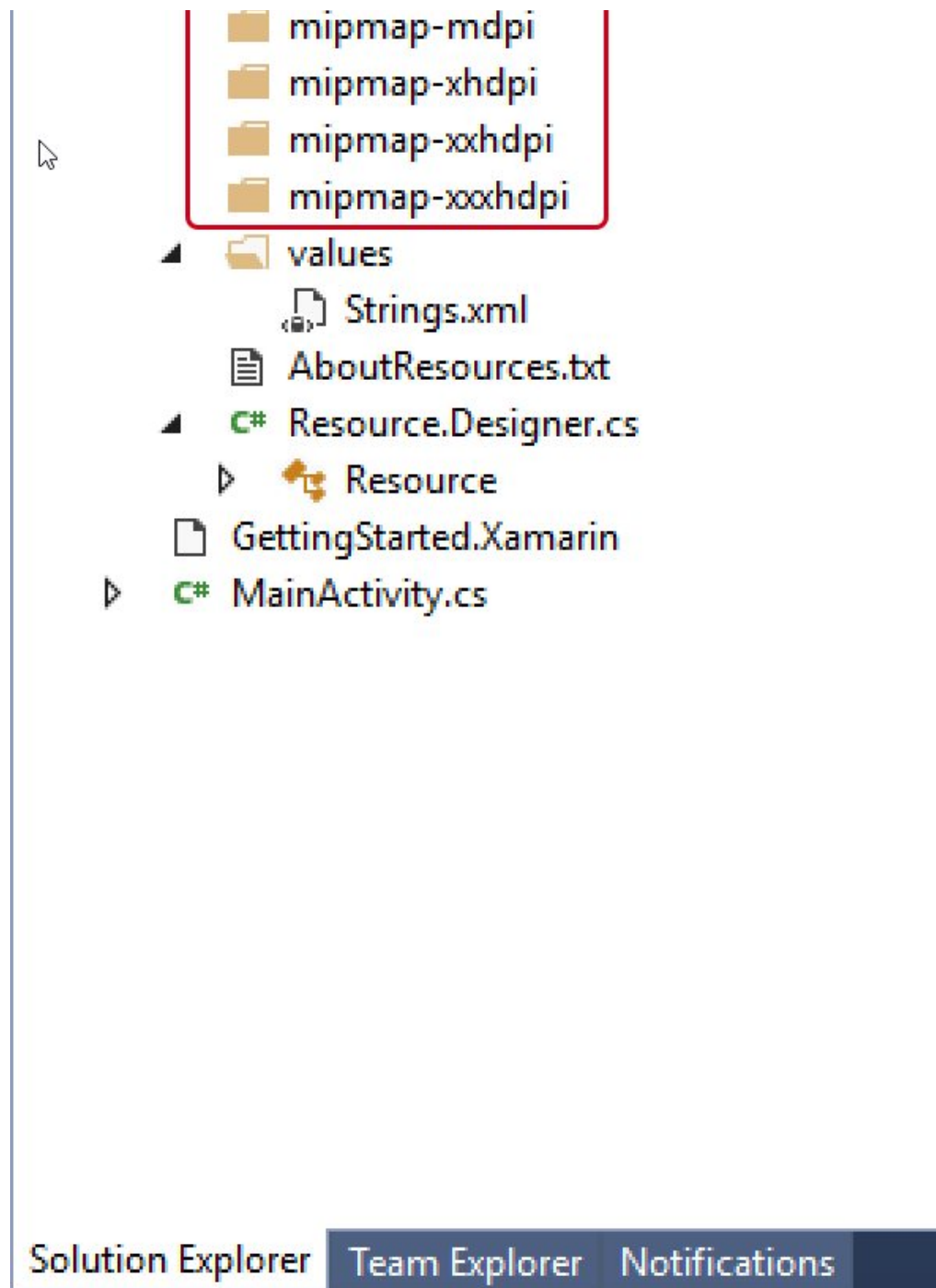




[[/ide]]

[[ide name="vs"]]





[[/ide]]

Android will pick the icon with the appropriate density:





Generate Custom Icons

Not everyone has a designer available to create the custom icons and launch images that an app needs to stand out. Here are several alternate approaches to generating custom app artwork:

[[ide name="xs"]]

- [Android Asset Studio](#) – A web-based, in-browser generator for all types of Android icons, with links to other useful community tools. It works best in Google Chrome.
- [Sketch 3](#) – Sketch is a Mac app for designing user interfaces, icons, and more. This is the app that was used to design the Xamarin App Icons and Launch Images set. Sketch 3 is available on the App Store and costs about \$80. You can try out the free [Sketch Tool](#) as well.
- [Pixelmator](#) – A versatile image editing app for Mac that costs about \$30.
- [Glyphish](#) – High-quality prebuilt icon sets for free download and purchase.
- [Fiverr](#) – Choose from a variety of designers to create an icon set for you, starting at \$5. Can be hit or miss but a good resource if you need icons designed on the fly.

[/ide]]

[[ide name="vs"]]

- [Android Asset Studio](#) – A web-based, in-browser generator for all types of Android icons, with links to other useful community tools. It works best in Google Chrome.
- Visual Studio – You can use this to create a simple icon set for your app directly in the IDE.
- [Glyphish](#) – High-quality prebuilt icon sets for free download and purchase.

- [Fiverr](#) – Choose from a variety of designers to create an icon set for you, starting at \$5. Can be hit or miss but a good resource if you need icons designed on the fly.

[[/ide]]

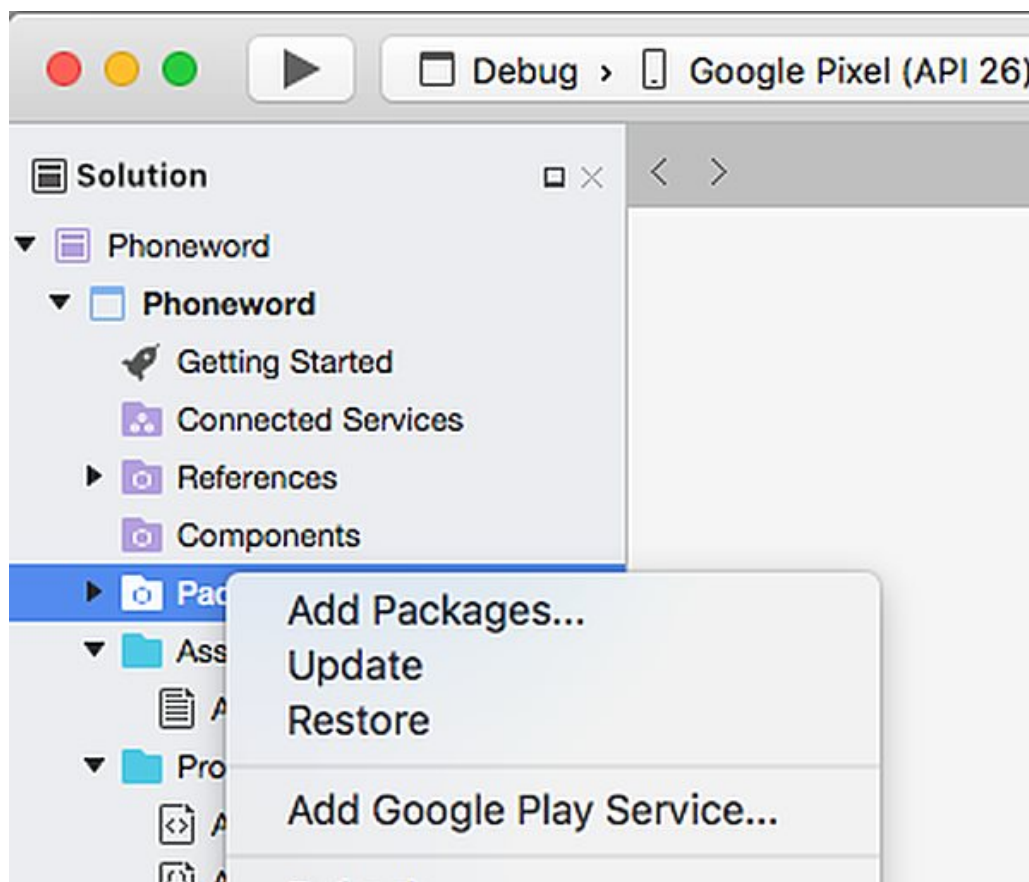
For more information about icon sizes and requirements, refer to the [Android Resources](#) guide.

[[ide name="xs"]]

Adding Google Play Services Packages

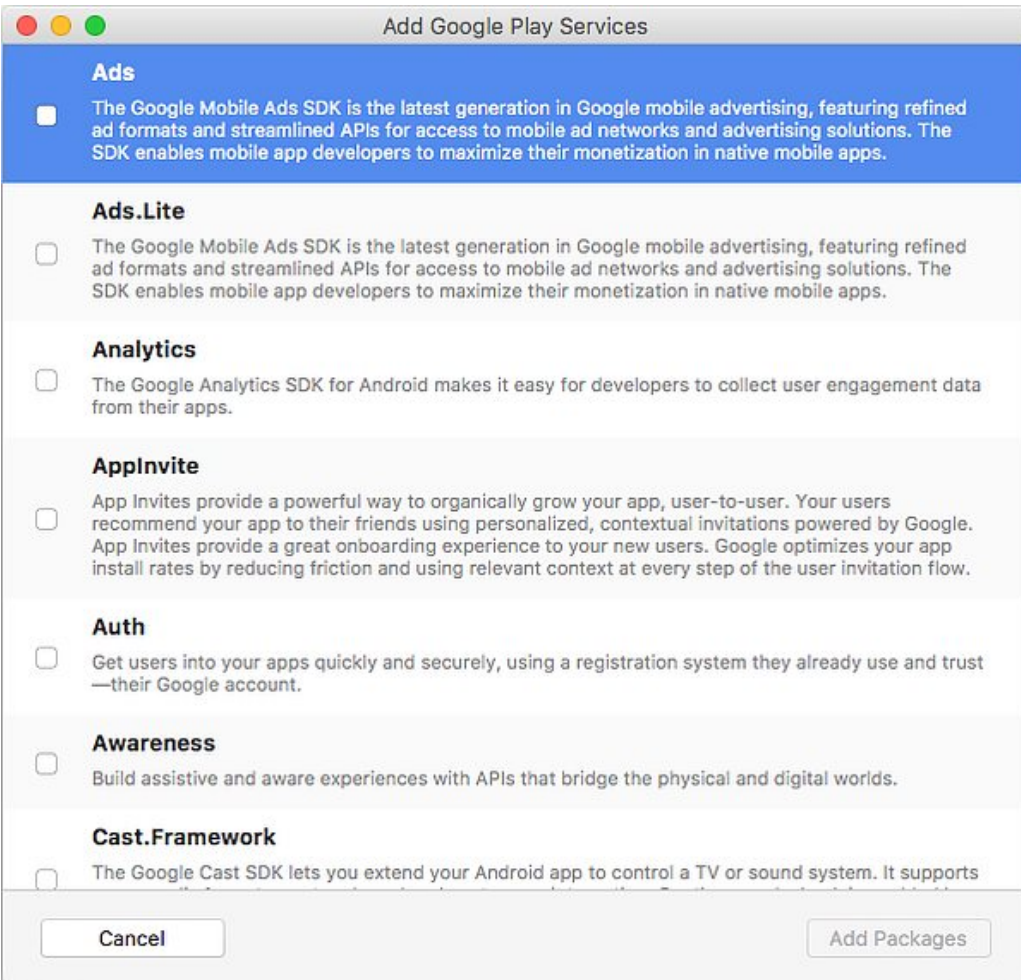
Google Play Services is a set of add-on libraries that allows Android developers to take advantage of the most recent features from Google such as Google Maps, Google Cloud Messaging, and in-app billing. Previously, bindings to all Google Play Services libraries were provided by Xamarin in the form of a single package – beginning with Visual Studio for Mac, a new project dialog is available for selecting which Google Play Services packages to include in your app.

To add one or more Google Play Service libraries, right-click the **Packages** node in your project tree and click **Add Google Play Service...**:

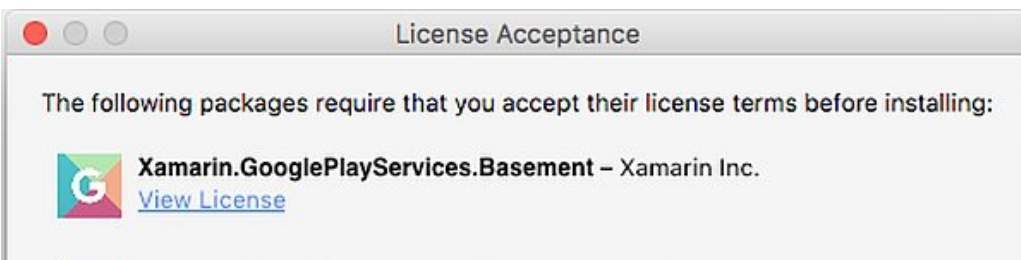


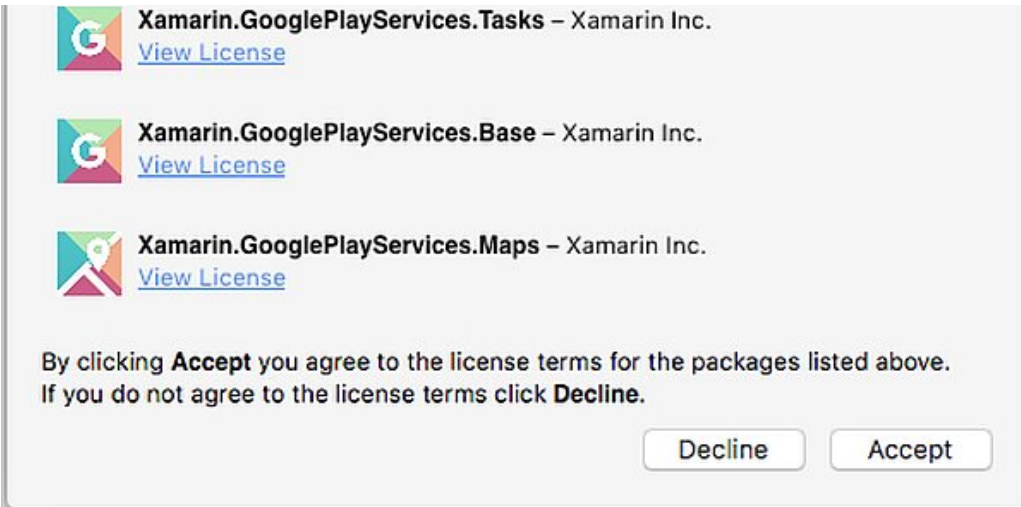


When the **Add Google Play Services** dialog is presented, select the packages (nugets) that you want to add to your project:



When you select a service and click **Add Package**, Visual Studio for Mac downloads and installs the package you select as well as any dependent Google Play Services packages that it requires. In some cases, you may see a **License Acceptance** dialog that requires you to click **Accept** before the packages are installed:





[[/ide]]

[[ide name="vs"]]

[[/ide]]

Summary

Congratulations! You should now have a solid understanding of the components of a Xamarin.Android application as well as the tools required to create it.

In the next tutorial of the *Getting Started* series, you will extend your application to handle multiple screens as you explore more advanced Android architecture and concepts.